

Azure AI Search vs SQL Server 2025 Hyperscale

Native vector + semantic search comparison · cost crossover model · migration map · East US pricing, May 2026

Build playbook (detailed) — execute each step end-to-end

Below is the operator-grade walkthrough for the 15-step build. Every step shows: **goal**, **prerequisites**, **commands / code**, **sample execution**, **validation check**, and the **common pitfalls**. All Azure CLI samples assume `az ≥ 2.60`. SQL samples assume Azure SQL Database Hyperscale (the SQL Server 2025 vector engine). Python samples assume Python 3.11 with `azure-identity`, `azure-ai-documentintelligence`, `openai`, `pyodbc`.

Variables used throughout: `RG=rg-search-prod` · `LOC=eastus` · `SRV=sql-search-prod` · `DB=searchdb` · `A0AI=aoai-search-prod` · `D0CAI=docai-search-prod`.

15 STEPS

- 01 Stand up the database
- 02 Design the schema
- 03 Replace indexers
- 04 Document cracking
- 05 Chunking
- 06 Embeddings
- 07 Vector index
- 08 Hybrid + RRF
- 09 Semantic re-ranker
- 10 Scoring & synonyms
- 11 Facets & suggesters
- 12 Observability
- 13 Security & identity

14 Agentic retrieval

15 Shadow cutover

01 Stand up the database

Goal: a production Hyperscale instance with HA + named-replica read scale-out and the preview switch flipped so DiskANN works.

PREREQUISITES

- Azure subscription with quota for *Premium Mem-Optimized Gen5* in your region
- VNet + subnet for the private endpoint
- An Entra group to act as the SQL admin (no SQL auth)

EXECUTE

PROVISION.SH

bash

```
RG=rg-search-prod
LOC=eastus
SRV=sql-search-prod
DB=searchdb
ADMIN_GROUP="grp-sql-search-admins"
ADMIN_GROUP_OID=$(az ad group show --group "$ADMIN_GROUP" --query id -o tsv)

az group create -n $RG -l $LOC

# Server with Entra-only auth
az sql server create -g $RG -n $SRV -l $LOC \
  --enable-ad-only-auth \
  --external-admin-principal-type Group \
  --external-admin-name "$ADMIN_GROUP" \
  --external-admin-sid $ADMIN_GROUP_OID

# Hyperscale database, Premium Mem-Opt for vector workloads
az sql db create -g $RG -s $SRV -n $DB \
  --edition Hyperscale \
  --family Gen5 \
```

```
--compute-model Provisioned \
--tier Hyperscale \
--capacity 16 \
--read-scale Enabled \
--high-availability-replica-count 1 \
--backup-storage-redundancy Zone \
--zone-redundant true

# Optional: one named replica for read scale-out (mirrors AI Search replica idea)
az sql db replica create -g $RG -s $SRV -n $DB \
  --partner-server $SRV --partner-database ${DB}-ro \
  --secondary-type Named --capacity 8 --family Gen5
```

FLIP THE PREVIEW SWITCH (REQUIRED FOR DISKANN, MAY 2026)

ENABLE-PREVIEW.SQL

sql

```
-- Run as a member of the Entra admin group
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;

SELECT name, value, is_value_default
FROM sys.database_scoped_configurations
WHERE name = 'PREVIEW_FEATURES';
```

SAMPLE OUTPUT

name	value	is_value_default
PREVIEW_FEATURES	1	0

Validation: `SELECT @@VERSION;` returns *Microsoft SQL Azure (RTM) – 16.x* with the 2025 build; `SELECT SERVERPROPERTY('EngineEdition') = 5` (Hyperscale).

Pitfall: picking Standard Gen5 vCore (not Premium Mem-Opt) means vector queries fall back to row-mode execution and DiskANN training is >5× slower. Spend the extra ~40% on Premium for any meaningful vector workload.

02 Design the schema

Goal: a normalized `documents` + `chunks` shape that supports vector search, full-text search, faceting, and audit — equivalent to an AI Search index with sub-document chunks.

EXECUTE

SCHEMA.SQL

sql

```
CREATE SCHEMA search;
GO

CREATE TABLE search.documents (
  doc_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID() PRIMARY KEY,
  source_uri      NVARCHAR(1024)   NOT NULL,
  source_etag     NVARCHAR(128)    NULL,
  title           NVARCHAR(512)    NULL,
  author          NVARCHAR(256)    NULL,
  tags            NVARCHAR(MAX)    NULL,          -- JSON array
  language        CHAR(2)          NULL,
  modified_at     DATETIME2(3)     NOT NULL,
  raw_text        NVARCHAR(MAX)    NULL,
  ingested_at     DATETIME2(3)     NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT UQ_documents_source UNIQUE (source_uri)
);

CREATE INDEX IX_documents_modified ON search.documents(modified_at);
CREATE INDEX IX_documents_author  ON search.documents(author);

CREATE TABLE search.chunks (
  chunk_id        BIGINT IDENTITY(1,1) PRIMARY KEY,
  doc_id          UNIQUEIDENTIFIER NOT NULL,
  ordinal         INT NOT NULL,
  text            NVARCHAR(MAX) NOT NULL,
  embedding       VECTOR(1536) NULL,             -- text-embedding-3-small
  embedding_model NVARCHAR(64) NULL,
  token_count     INT NULL,
  created_at      DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_chunks_doc FOREIGN KEY (doc_id)
```

```
REFERENCES search.documents(doc_id) ON DELETE CASCADE
);

CREATE INDEX IX_chunks_doc ON search.chunks(doc_id, ordinal);

-- Full-text catalog for lexical / BM25-like search
CREATE FULLTEXT CATALOG ft_search AS DEFAULT;
CREATE FULLTEXT INDEX ON search.chunks(text LANGUAGE 1033)
KEY INDEX PK__chunks__... ON ft_search WITH CHANGE_TRACKING AUTO;
```

Validation:

```
SELECT t.name, c.name, ty.name AS dtype, c.max_length
FROM sys.tables t JOIN sys.columns c ON c.object_id = t.object_id
JOIN sys.types ty ON ty.user_type_id = c.user_type_id
WHERE t.name IN ('documents','chunks') ORDER BY t.name, c.column_id;
```

Expect the `embedding` column to show type `vector` with `max_length = 6148` (1536 dims × 4 bytes + 4-byte header).

Pitfall: `VECTOR(N)` in SQL Server 2025 caps at **1 998 dimensions**. `text-embedding-3-large` (3072 dims) *does not fit*. Use `text-embedding-3-small` (1536 dims) or truncate large embeddings with the `AOAI_dimensions` parameter to stay under the cap.

03 Replace indexers — build an ingestion pipeline

Goal: reliably pick up new / changed source documents and land them in `search.documents`, then trigger chunk + embed downstream. AI Search indexers are a managed scheduler with built-in change tracking — here you own that.

REFERENCE ARCHITECTURE

- **Trigger:** Azure Data Factory tumbling-window trigger every 15 min, or Event Grid for instant ingestion from Blob.
- **Pattern:** high-water mark (`modified_at > LAST_RUN_TS`) for SQL/CDC sources; ETag/ `LastModified` headers for Blob/ADLS; `_ts` filter for Cosmos DB.
- **State:** store last-run-cursor in a small bookkeeping table `search.ingestion_state`.

FUNCTION_APP.PY

python

```

import os, hashlib, logging
import azure.functions as func
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient
import pyodbc

app = func.FunctionApp()

SQL_CONN = os.environ["SQL_CONN"]    # uses Azure AD interactive / MSI
BLOB_URL = os.environ["BLOB_URL"]

@app.schedule(schedule="0 */15 * * * *", arg_name="timer")
def ingest(timer: func.TimerRequest) -> None:
    cred = DefaultAzureCredential()
    svc = BlobServiceClient(account_url=BLOB_URL, credential=cred)
    container = svc.get_container_client("corpus")

    with pyodbc.connect(SQL_CONN, autocommit=False) as cn:
        cur = cn.cursor()
        cur.execute("SELECT last_run FROM search.ingestion_state WHERE name='blob-corpus'")
        last_run = cur.fetchone()[0]

    upserts = 0
    for blob in container.list_blobs():
        if blob.last_modified <= last_run:
            continue
        etag = blob.etag.strip('"')
        cur.execute("""
            MERGE search.documents AS t
            USING (SELECT ? AS source_uri, ? AS source_etag,
                        ? AS title, ? AS modified_at) AS s
            ON t.source_uri = s.source_uri
            WHEN MATCHED AND t.source_etag <> s.source_etag THEN
                UPDATE SET source_etag = s.source_etag,
                           modified_at = s.modified_at,
                           raw_text = NULL -- force re-crack
            WHEN NOT MATCHED THEN
                INSERT (source_uri, source_etag, title, modified_at)
                VALUES (s.source_uri, s.source_etag, s.title, s.modified_at);
        """)

```

```
"""", f"{BLOB_URL}/corpus/{blob.name}", etag, blob.name, blob.last_modified)
upserts += 1
```

```
cur.execute("UPDATE search.ingestion_state SET last_run = SYSUTCDATETIME() WHERE name='blob-corpus'")
cn.commit()
logging.info(f"Ingest done: {upserts} upserts")
```

SAMPLE EXECUTION

```
2026-05-21T02:00:01Z [INFO] Ingest done: 137 upserts
2026-05-21T02:15:01Z [INFO] Ingest done: 12 upserts
2026-05-21T02:30:01Z [INFO] Ingest done: 0 upserts
```

Validation: after one run, `SELECT COUNT(*) FROM search.documents WHERE raw_text IS NULL;` shows the queue for Step 4 (cracking).

Pitfall: resetting the high-water mark to "0" silently re-ingests everything and re-embeds (Step 6) at full cost. Always re-ingest by deleting specific rows from `ingestion_state` AND zeroing `raw_text` for the affected docs only.

04 Replace document cracking

Goal: extract text from PDFs, Office docs, scanned images. Equivalent of the built-in cracking skill in AI Search.

PROVISION DOCUMENT INTELLIGENCE

DOCAI.SH

bash

```
az cognitiveservices account create \
  -g $RG -n $DOCAI -l $LOC \
  --kind FormRecognizer --sku S0 \
  --custom-domain $DOCAI \
  --assign-identity
```

CRACKER FUNCTION (DRAINS `RAW_TEXT IS NULL` QUEUE)

CRACK.PY

python

```

from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.identity import DefaultAzureCredential
import pyodbc, os

cred = DefaultAzureCredential()
di = DocumentIntelligenceClient(
    endpoint=os.environ["DOCAI_ENDPOINT"], credential=cred)

cn = pyodbc.connect(os.environ["SQL_CONN"], autoccommit=False)
cur = cn.cursor()
cur.execute("""
    SELECT TOP 50 doc_id, source_uri
    FROM search.documents
    WHERE raw_text IS NULL
    ORDER BY modified_at
""")
rows = cur.fetchall()

for doc_id, uri in rows:
    poller = di.begin_analyze_document(
        "prebuilt-layout", AnalyzeDocumentRequest(url_source=uri))
    result = poller.result()
    text = "\n\n".join(p.content for p in result.paragraphs)
    cur.execute("UPDATE search.documents SET raw_text = ? WHERE doc_id = ?",
                text, doc_id)

cn.commit()
print(f"Cracked {len(rows)} docs")

```

SAMPLE EXECUTION

```

$ python crack.py
Cracked 50 docs
$ python crack.py
Cracked 50 docs
$ python crack.py
Cracked 37 docs

```


Pitfall: `prebuilt-layout` costs ~\$10/1K pages. For pure-text PDFs use `prebuilt-read` (~\$1.50/1K). Profile your corpus first — running layout on a 1M-page corpus = \$10K just to crack.

05 Replace the Text Split skill — chunking

Goal: split each document's `raw_text` into overlapping chunks that fit your embedding model's token window, persisted as rows in `search.chunks`.

CHUNK.PY

python

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
import pyodbc, tiktoken, os

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,          # tokens
    chunk_overlap=120,
    length_function=lambda s: len(tiktoken.get_encoding("cl100k_base").encode(s)),
    separators=["\n\n", "\n", ". ", " ", ""])

cn = pyodbc.connect(os.environ["SQL_CONN"], autocommit=False)
cur = cn.cursor()

# Docs that have text but no chunks yet
cur.execute("""
    SELECT d.doc_id, d.raw_text
    FROM search.documents d
    WHERE d.raw_text IS NOT NULL
    AND NOT EXISTS (SELECT 1 FROM search.chunks c WHERE c.doc_id = d.doc_id)
""")

for doc_id, text in cur.fetchall():
    for i, chunk in enumerate(splitter.split_text(text)):
        cur.execute("""
            INSERT INTO search.chunks (doc_id, ordinal, text, token_count)
            VALUES (?, ?, ?, ?)""",
            doc_id, i, chunk,
```

```
len(tiktoken.get_encoding("cl100k_base").encode(chunk)))
cn.commit()
```

SAMPLE ROW AFTER RUN

chunk_id	doc_id	ordinal	token_count	text
1	a2f9...c1	0	742	"Members enrolled in HMO plans must..."
2	a2f9...c1	1	788	"...selection of primary care provider..."
3	a2f9...c1	2	611	"...annual deductibles apply to..."

Pitfall: chunking on raw character count instead of tokens means ~30% of your chunks will exceed the embedding model's 8 191 token window and silently truncate. Always tokenize.

06 Replace integrated vectorization — embeddings

Goal: embed every `chunks.text` with Azure OpenAI and store the vector. AI Search calls AOAI for you; here you own batching, retries, throttling.

OPTION A — EXTERNAL BATCHER (PREFERRED FOR THROUGHPUT)

EMBED.PY

python

```
from openai import AzureOpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
import pyodbc, os, struct

cred = DefaultAzureCredential()
token = get_bearer_token_provider(cred, "https://cognitiveservices.azure.com/.default")
client = AzureOpenAI(
    azure_endpoint=os.environ["AOAI_ENDPOINT"],
    azure_ad_token_provider=token,
    api_version="2024-10-21")
MODEL = "text-embedding-3-small" # 1536 dims, fits in VECTOR(1536)

cn = pyodbc.connect(os.environ["SQL_CONN"], autocommit=False)
```

```

cur = cn.cursor()

BATCH = 64
while True:
    cur.execute("""
        SELECT TOP (?) chunk_id, text
        FROM search.chunks WHERE embedding IS NULL ORDER BY chunk_id
        """, BATCH)
    rows = cur.fetchall()
    if not rows: break

    resp = client.embeddings.create(model=MODEL, input=[r.text for r in rows])
    for r, e in zip(rows, resp.data):
        # SQL Server accepts vector literal as JSON array string
        vec_json = "[" + ",".join(f"{v:.6f}" for v in e.embedding) + "]"
        cur.execute("""
            UPDATE search.chunks
            SET embedding = CAST(? AS VECTOR(1536)),
                embedding_model = ?
            WHERE chunk_id = ?
            """, vec_json, MODEL, r.chunk_id)
    cn.commit()
    print(f"Embedded batch of {len(rows)}")

```

OPTION B — IN-DATABASE VIA `SP_INVOKE_EXTERNAL_REST_ENDPOINT`

EMBED_IN_DB.SQL

sql

```

-- One-time: managed identity + scoped credential to AOAI
CREATE DATABASE SCOPED CREDENTIAL AOAI Cred
WITH IDENTITY = 'Managed Identity';

CREATE OR ALTER PROCEDURE search.embed_chunk @chunk_id BIGINT
AS
BEGIN
    DECLARE @text NVARCHAR(MAX), @payload NVARCHAR(MAX), @resp NVARCHAR(MAX);
    SELECT @text = text FROM search.chunks WHERE chunk_id = @chunk_id;

    SET @payload = JSON_OBJECT('input': @text, 'model': 'text-embedding-3-small');

```

```

EXEC sp_invoke_external_rest_endpoint
    @url      = 'https://aoai-search-prod.openai.azure.com/openai/deployments/te3s/embeddings?api-version=2024-10-21',
    @method   = 'POST',
    @payload  = @payload,
    @credential = AOAICred,
    @response = @resp OUTPUT;

DECLARE @vec NVARCHAR(MAX) =
    (SELECT value FROM OPENJSON(@resp, '$.result.data[0].embedding'));

UPDATE search.chunks
    SET embedding = CAST(@vec AS VECTOR(1536)),
        embedding_model = 'text-embedding-3-small'
    WHERE chunk_id = @chunk_id;
END

```

SAMPLE EXECUTION

```

$ python embed.py
Embedded batch of 64
Embedded batch of 64
Embedded batch of 64
...
Embedded batch of 17

```

Validation: `SELECT VECTOR_DIMS(embedding) FROM search.chunks WHERE chunk_id = 1;` returns **1536**.

Pitfall A: Option B (per-row stored proc) hits AOAI request-per-minute quotas FAST at scale. Use it only for trickle re-embeds; do bulk in Option A.

Pitfall B: if you re-embed with a new model, set `embedding_model` too and consider re-indexing — DiskANN's centroids are model-specific.

Goal: create the DiskANN ANN index so KNN search is sub-second on millions of vectors. Without it, you're doing brute-force scans.

VECTOR-INDEX.SQL

sql

```
-- Confirm preview features are on (Step 1)
SELECT value FROM sys.database_scoped_configurations WHERE name = 'PREVIEW_FEATURES';

CREATE VECTOR INDEX IX_chunks_emb_diskann
ON search.chunks (embedding)
WITH (
    METRIC = 'cosine',
    TYPE   = 'diskann',
    MAXDOP = 8
);

-- Track build progress (build is async on Hyperscale)
SELECT name, type_desc, is_disabled, has_filter
FROM sys.indexes WHERE object_id = OBJECT_ID('search.chunks');

SELECT * FROM sys.dm_db_xtp_hash_index_stats; -- runtime stats
```

SAMPLE EXECUTION TIMELINE

```
2026-05-21 03:00:00  CREATE VECTOR INDEX submitted (~ 4.2M rows)
2026-05-21 03:00:01  command returns immediately
2026-05-21 03:11:37  index online - sys.indexes.is_disabled = 0
total cost: 11m 36s wall, ~22 vCore-minutes
```

Smoke test:

KNN-SMOKE.SQL

sql

```
DECLARE @q VECTOR(1536) = (
    SELECT embedding FROM search.chunks WHERE chunk_id = 1);

SELECT TOP 5 chunk_id,
    VECTOR_DISTANCE('cosine', embedding, @q) AS dist
```

```
FROM search.chunks
ORDER BY VECTOR_DISTANCE('cosine', embedding, @q);
```

Top-1 distance should be 0.0 (the query itself); top-5 returned in < 100ms on a warm cache.

Pitfall: DiskANN at GA is **single-replica** and is dropped + rebuilt on full rebuilds — no online maintenance. Plan a maintenance window for any large delete-heavy rebuild. Also: it's still **public preview** in May 2026; do not rely on its SLA without compensating controls.

08 Re-implement hybrid retrieval with RRF

Goal: a single stored proc that does *full-text + vector KNN + Reciprocal Rank Fusion*, returning a ranked top-K. This is what AI Search gives you for free with `queryType=semantic, vectorQueries=[...]`.

HYBRID_SEARCH.SQL

sql

```
CREATE OR ALTER PROCEDURE search.hybrid_search
    @query_text NVARCHAR(1000),
    @query_vec  VECTOR(1536),
    @top_k      INT = 50,
    @rrf_k      INT = 60          -- RRF smoothing constant; 60 is the canonical default
AS
BEGIN
    ;WITH
    lex AS (
        SELECT TOP (@top_k) c.chunk_id,
            ROW_NUMBER() OVER (ORDER BY KEY_TBL.RANK DESC) AS r
        FROM search.chunks c
        INNER JOIN CONTAINSTABLE(search.chunks, text, @query_text, @top_k) AS KEY_TBL
            ON c.chunk_id = KEY_TBL.[KEY]
    ),
    vec AS (
        SELECT TOP (@top_k) chunk_id,
            ROW_NUMBER() OVER (
                ORDER BY VECTOR_DISTANCE('cosine', embedding, @query_vec)) AS r
        FROM search.chunks
        WHERE embedding IS NOT NULL
```

```

),
fused AS (
  SELECT chunk_id, SUM(1.0 / (@rrf_k + r)) AS rrf_score
  FROM (SELECT * FROM lex UNION ALL SELECT * FROM vec) u
  GROUP BY chunk_id
)
SELECT TOP (@top_k)
  f.chunk_id, f.rrf_score,
  c.text, d.title, d.source_uri, d.author, d.modified_at
FROM fused f
JOIN search.chunks c ON c.chunk_id = f.chunk_id
JOIN search.documents d ON d.doc_id = c.doc_id
ORDER BY f.rrf_score DESC;
END

```

SAMPLE CALL

CALL-HYBRID.SQL

sql

```

DECLARE @qv VECTOR(1536) = /* embed("annual deductible for HMO") via Step 6 */ ...;
EXEC search.hybrid_search
  @query_text = N'annual deductible HMO',
  @query_vec  = @qv,
  @top_k      = 25;

```

SAMPLE OUTPUT

chunk_id	rrf_score	title	source_uri
8842	0.03145	Member Benefits 2026	.../mbk-2026.pdf
8841	0.02877	Member Benefits 2026	.../mbk-2026.pdf
14093	0.02560	HMO Plan Summary	.../hmo-summary.pdf
...			

Pitfall: RRF requires both ranked lists to use the SAME row identifier. If you join on `doc_id` in one branch and `chunk_id` in the other, fusion is meaningless. Fix scope first.

09 Build a semantic re-ranker — the biggest gap

Goal: rescore the top-50 from Step 8 with a cross-encoder to get AI Search-quality semantic ranking. Highest cost, biggest quality lift.

OPTION A — SELF-HOSTED CROSS-ENCODER (RECOMMENDED FOR COST)

RERANKER_APP.PY

python

```
from fastapi import FastAPI
from sentence_transformers import CrossEncoder
from pydantic import BaseModel

model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-12-v2", device="cuda")
app = FastAPI()

class Req(BaseModel):
    query: str
    passages: list[str]

@app.post("/rerank")
def rerank(r: Req):
    scores = model.predict([(r.query, p) for p in r.passages]).tolist()
    order = sorted(range(len(r.passages)), key=lambda i: -scores[i])
    return {"order": order, "scores": [scores[i] for i in order]}
```

DEPLOY.SH

bash

```
az containerapp create \
  -g $RG -n reranker \
  --environment search-env \
  --image myacr.azurecr.io/reranker:1.0 \
  --workload-profile-name gpu-nc24 \
  --min-replicas 1 --max-replicas 4 \
  --ingress internal --target-port 8000
```

OPTION B — GPT-4O-MINI SCORER (HIGHER QUALITY, ~20× COST)

GPT_RERANK.PY

python


```
prompt = """You are a relevance grader. Given a query and a passage, return
an integer 0..10 for how relevant the passage is. Output only the integer."""
```

```
scores = []
for p in passages:
    r = client.chat.completions.create(model="gpt-4o-mini",
        messages=[{"role":"system","content":prompt},
            {"role":"user","content":f"QUERY: {query}\nPASSAGE: {p}" }],
        max_tokens=2, temperature=0)
    scores.append(int(r.choices[0].message.content.strip()))
```

WIRE INTO THE APP TIER

SEARCH_API.PY

python

```
def search(query: str, top_k: int = 10):
    qv = embed(query)
    candidates = sql.exec("EXEC search.hybrid_search ?, ?, 50", query, qv)
    rerank = httpx.post("https://reranker/rerank",
        json={"query": query, "passages": [c.text for c in candidates]}).json()
    return [candidates[i] for i in rerank["order"][:top_k]]
```

Pitfall: running the cross-encoder on the FULL result set instead of top-50 destroys latency. The whole point is "cheap recall (hybrid) + expensive precision (reranker on 50 candidates)."

10 Recreate scoring profiles and synonyms

Goal: field-weighting, freshness boosts, tag boosts, and synonym expansion — features AI Search exposes as JSON config.

SCORING EXPRESSION IN T-SQL

SCORING-PROFILE.SQL

sql

```
-- Compose a final score from RRF + freshness boost + tag boost
SELECT TOP 25
    f.chunk_id, f.rrf_score,
```

```

freshness_boost = CAST(
    1.0 / (1 + DATEDIFF(DAY, d.modified_at, SYSUTCDATETIME()) / 30.0) AS FLOAT),
tag_boost = CASE
    WHEN JSON_VALUE(d.tags, '$[0]') IN ('benefits','formulary') THEN 1.3
    ELSE 1.0 END,
final_score = f.rrf_score
    * (1 + 0.4 * (1.0 / (1 + DATEDIFF(DAY, d.modified_at, SYSUTCDATETIME()) / 30.0)))
    * (CASE WHEN JSON_VALUE(d.tags, '$[0]') IN ('benefits','formulary')
        THEN 1.3 ELSE 1.0 END)

FROM fused f
JOIN search.documents d ON ...
ORDER BY final_score DESC;

```

SYNONYMS — FTS THESAURUS

TSENU.XML

xml

```

<XML ID="Microsoft Search Thesaurus">
  <thesaurus xmlns="x-schema:tsSchema.xml">
    <diacritics_sensitive>0</diacritics_sensitive>
    <expansion>
      <sub>HMO</sub><sub>Health Maintenance Organization</sub>
    </expansion>
    <expansion>
      <sub>PCP</sub><sub>primary care provider</sub><sub>primary care physician</sub>
    </expansion>
  </thesaurus>
</XML>

```

LOAD-THESAURUS.SQL

sql

```

EXEC sys.sp_fulltext_load_thesaurus_file 1033, @LoadOnly = 0;
-- Re-issue queries; FORMSOF(THESAURUS, 'HMO') now also matches 'Health Maintenance Organization'

```

Goal: the "filter pills" + autocomplete dropdown UX. AI Search ships these as first-class endpoints; you build them.

FACETS

FACETS.SQL

sql

```
-- Counts by author and language, scoped to a result set CTE
WITH hits AS (
  SELECT d.doc_id, d.author, d.language
  FROM search.documents d JOIN search.chunks c ON c.doc_id = d.doc_id
  WHERE c.chunk_id IN (/* result chunk_ids from Step 8 */)
)
SELECT 'author' AS facet, author AS value, COUNT(*) AS cnt
FROM hits GROUP BY author
UNION ALL
SELECT 'language', language, COUNT(*) FROM hits GROUP BY language
ORDER BY facet, cnt DESC;
```

SUGGESTER / AUTOCOMPLETE

SUGGESTER.SQL

sql

```
-- Materialized view of distinct title prefixes
CREATE TABLE search.title_prefixes (prefix NVARCHAR(64) PRIMARY KEY);
INSERT INTO search.title_prefixes
SELECT DISTINCT LOWER(LEFT(title, n))
FROM search.documents
CROSS APPLY (VALUES (3), (5), (8), (12), (20)) AS p(n)
WHERE title IS NOT NULL;

-- Autocomplete query
SELECT TOP 10 prefix
FROM search.title_prefixes
WHERE prefix LIKE @typed + '%'
ORDER BY prefix;
```

Pitfall: querying facets against the full `chunks` table for every search request is slow. Always scope to the candidate set returned by Step 8.

12 Wire up observability & query telemetry

Goal: per-query latency, candidate counts, click-through, and a relevance dashboard. AI Search Insights is gone — you stitch it together.

ENABLE PLATFORM DIAGNOSTICS

DIAG.SH

bash

```
LAW=law-search-prod
az monitor log-analytics workspace create -g $RG -n $LAW

az monitor diagnostic-settings create \
  --resource $(az sql db show -g $RG -s $SRV -n $DB --query id -o tsv) \
  --name diag-all \
  --workspace $LAW \
  --logs '[{"category":"QueryStoreRuntimeStatistics","enabled":true},
          {"category":"AutomaticTuning","enabled":true},
          {"category":"Errors","enabled":true}]' \
  --metrics '[{"category":"Basic","enabled":true}]'
```

APP-TIER QUERY LOG TABLE

TELEMETRY.SQL

sql

```
CREATE TABLE search.query_log (
  query_id      UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
  ts            DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
  user_id       NVARCHAR(128),
  query_text    NVARCHAR(1000),
  candidates    INT,
  top_chunk_id  BIGINT,
  rrf_top       FLOAT,
  rerank_top    FLOAT,
  total_ms      INT,
  embed_ms      INT,
  hybrid_ms     INT,
  rerank_ms     INT,
  clicked_at    DATETIME2(3) NULL,
```

```
clicked_rank INT NULL
);
```

RELEVANCE DASHBOARD (KQL ON LOG ANALYTICS)

RELEVANCE.KQL

sql

```
// p50/p95 latency by hour
AppRequests
| where Name == "POST /search"
| summarize p50=percentile(DurationMs,50), p95=percentile(DurationMs,95) by bin(TimeGenerated,1h)
| render timechart

// CTR at rank 1
customEvents
| where name == "search_click"
| summarize ctr = todouble(countif(toint(customDimensions.clicked_rank)==1))/count()
by bin(timestamp,1d)
```

13 Security, network, and identity

Goal: the controls a regulated workload (HIPAA / HITRUST / FedRAMP) needs — TDE+CMK, private endpoint, Entra-only, RLS, column-level encryption.

TDE WITH CUSTOMER-MANAGED KEY

CMK.SH

bash

```
KV=kv-search-prod
KEY=cmk-search-2026

az keyvault create -g $RG -n $KV -l $LOC --enable-purge-protection true
az keyvault key create --vault-name $KV -n $KEY --key RSA --size 3072

# Grant the SQL server's managed identity wrap/unwrap/get
az keyvault set-policy -n $KV \
```

```
--object-id $(az sql server show -g $RG -n $SRV --query identity.principalId -o tsv) \
--key-permissions wrapKey unwrapKey get

KEY_URI=$(az keyvault key show --vault-name $KV -n $KEY --query key.kid -o tsv)

az sql server tde-key set -g $RG -s $SRV --server-key-type AzureKeyVault \
--kid $KEY_URI

az sql db tde set -g $RG -s $SRV -n $DB --status Enabled
```

PRIVATE ENDPOINT

PE.SH

bash

```
az network private-endpoint create -g $RG -n pe-sql-search \
--vnet-name vnet-search --subnet snet-data \
--private-connection-resource-id $(az sql server show -g $RG -n $SRV --query id -o tsv) \
--group-id sqlServer \
--connection-name pe-sql-search-conn

az sql server update -g $RG -n $SRV --enable-public-network false
```

ROW-LEVEL SECURITY (MULTI-TENANT PATTERN)

RLS.SQL

sql

```
CREATE SCHEMA sec;
GO
CREATE FUNCTION sec.fn_tenant_predicate(@tenant NVARCHAR(64))
RETURNS TABLE WITH SCHEMABINDING AS
RETURN SELECT 1 AS ok
WHERE @tenant = CAST(SESSION_CONTEXT(N'tenant_id') AS NVARCHAR(64));
GO

ALTER TABLE search.documents ADD tenant_id NVARCHAR(64) NOT NULL DEFAULT 'public';

CREATE SECURITY POLICY sec.documents_filter
ADD FILTER PREDICATE sec.fn_tenant_predicate(tenant_id) ON search.documents,
```

```
ADD BLOCK PREDICATE sec.fn_tenant_predicate(tenant_id) ON search.documents
WITH (STATE = ON);
```

APP SETS THE TENANT BEFORE EVERY QUERY

APP.PY

python

```
cur.execute("EXEC sp_set_session_context @key=N'tenant_id', @value=?", tenant_id)
# All subsequent queries on search.documents are filtered automatically
```

14 If you used agentic retrieval — rebuild it

Goal: multi-turn query planning + reasoning + retrieval. AI Search calls this "agentic retrieval"; you rebuild it in your app tier with an LLM orchestrator.

AGENTIC.PY

python

```
from semantic_kernel import Kernel
from semantic_kernel.connectors.ai.open_ai import AzureChatCompletion
from semantic_kernel.functions import kernel_function

kernel = Kernel()
kernel.add_service(AzureChatCompletion(deployment_name="gpt-4o",
    endpoint=os.environ["AOAI_ENDPOINT"], ad_token_provider=token))

class SearchPlugin:
    @kernel_function(description="Hybrid + rerank search against SQL")
    def search(self, query: str, top_k: int = 10) -> str:
        return json.dumps(do_hybrid_search(query, top_k)) # wraps Step 9

kernel.add_plugin(SearchPlugin(), "kb")

PLANNER = """You are a healthcare claims expert. Decompose the user's question
into 1-3 search sub-queries against the knowledge base, run them via the
kb.search function, then synthesize a grounded answer with citations."""
```



```
def ndcg(judged, returned, k=10):
    dcg = sum(judged.get(d, 0) / math.log2(i + 2) for i, d in enumerate(returned[:k]))
    ideal = sorted(judged.values(), reverse=True)[:k]
    idcg = sum(g / math.log2(i + 2) for i, g in enumerate(ideal))
    return dcg / idcg if idcg else 0.0

with open("query_judgments.jsonl") as f:
    rows = [json.loads(l) for l in f]

ais_ndcg, sql_ndcg = [], []
for r in rows:
    q, judged = r["query"], r["judgments"] # {doc_id: rel (0-3)}
    ais_top = [d["doc_id"] for d in ais.search(q, top=10)]
    sql_top = [d["doc_id"] for d in do_hybrid_search(q, top_k=10)]
    ais_ndcg.append(ndcg(judged, ais_top))
    sql_ndcg.append(ndcg(judged, sql_top))

print(f"AI Search NDCG@10: {statistics.mean(ais_ndcg):.4f}")
print(f"SQL Hybrid NDCG@10: {statistics.mean(sql_ndcg):.4f}")
print(f"Delta: {(statistics.mean(sql_ndcg) - statistics.mean(ais_ndcg)) * 100:+.2f} pts")
```

SAMPLE EXECUTION

```
$ python shadow.py
Evaluating 1,200 queries...
AI Search NDCG@10: 0.7421
SQL Hybrid NDCG@10: 0.7388
Delta: -0.33 pts

Verdict: within 0.5 pts -> GO for cutover
```

CUTOVER PLAN

1. **Day -14:** deploy SQL stack to prod, dual-write ingestion (writes go to BOTH AI Search and SQL).
2. **Day -7:** dual-read with shadow eval (above) running on every real query in the background; log deltas.
3. **Day 0:** flip read traffic to SQL via feature flag (1% → 10% → 50% → 100% over 48h).
4. **Day +7:** stop dual-write; decommission the AI Search service.

Go / no-go gate: SQL NDCG@10 within 0.5 pts of AI Search AND p95 latency < AI Search p95 + 100ms. Anything worse, do not cut over — invest more in the re-ranker (Step 9).